

什么是容器：从基础到进阶的全面介绍

发布于 2024-05-28 08:26:16

👁 4.4K

💬 0

🔗 2

代码可运行

△ 举报

文章被收录于专栏：[CSDN社区搬运](#)

关联问题

换一批

容器技术的核心概念是什么？

如何实现容器的编排和管理？

容器与传统虚拟机有哪些区别？

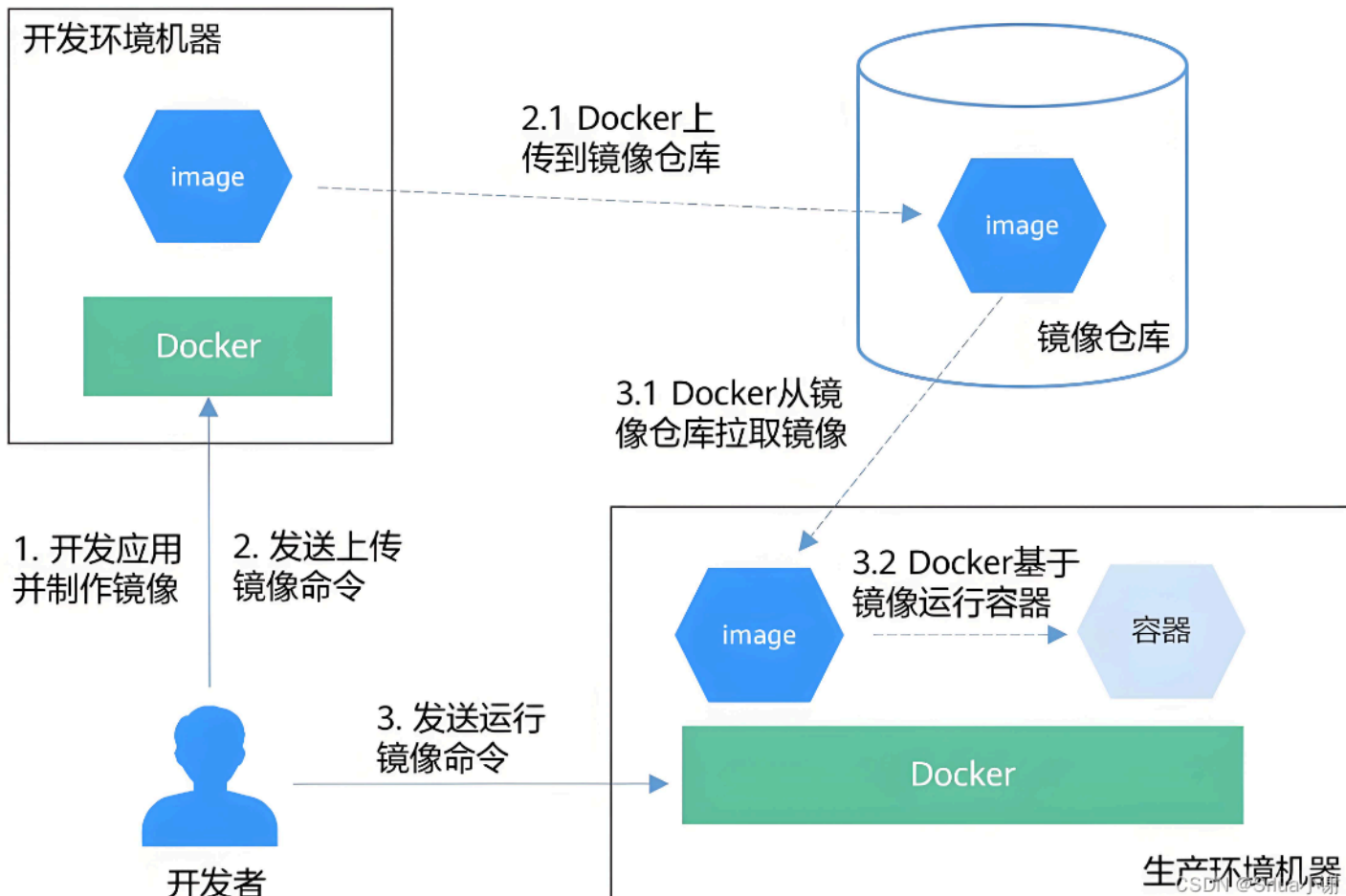
什么是容器：从基础到进阶的全面介绍



容器 技术是现代 **软件开发** 和部署中的一种革命性工具，它提供了一种轻量级且高效的方式来打包和运行应用程序。本文将深入探讨容器的基本概念、其优点、如何使用容器、以及在实际应用中的一些高级用法。我们将通过分点讲解，逐步带您了解这一技术的全貌。

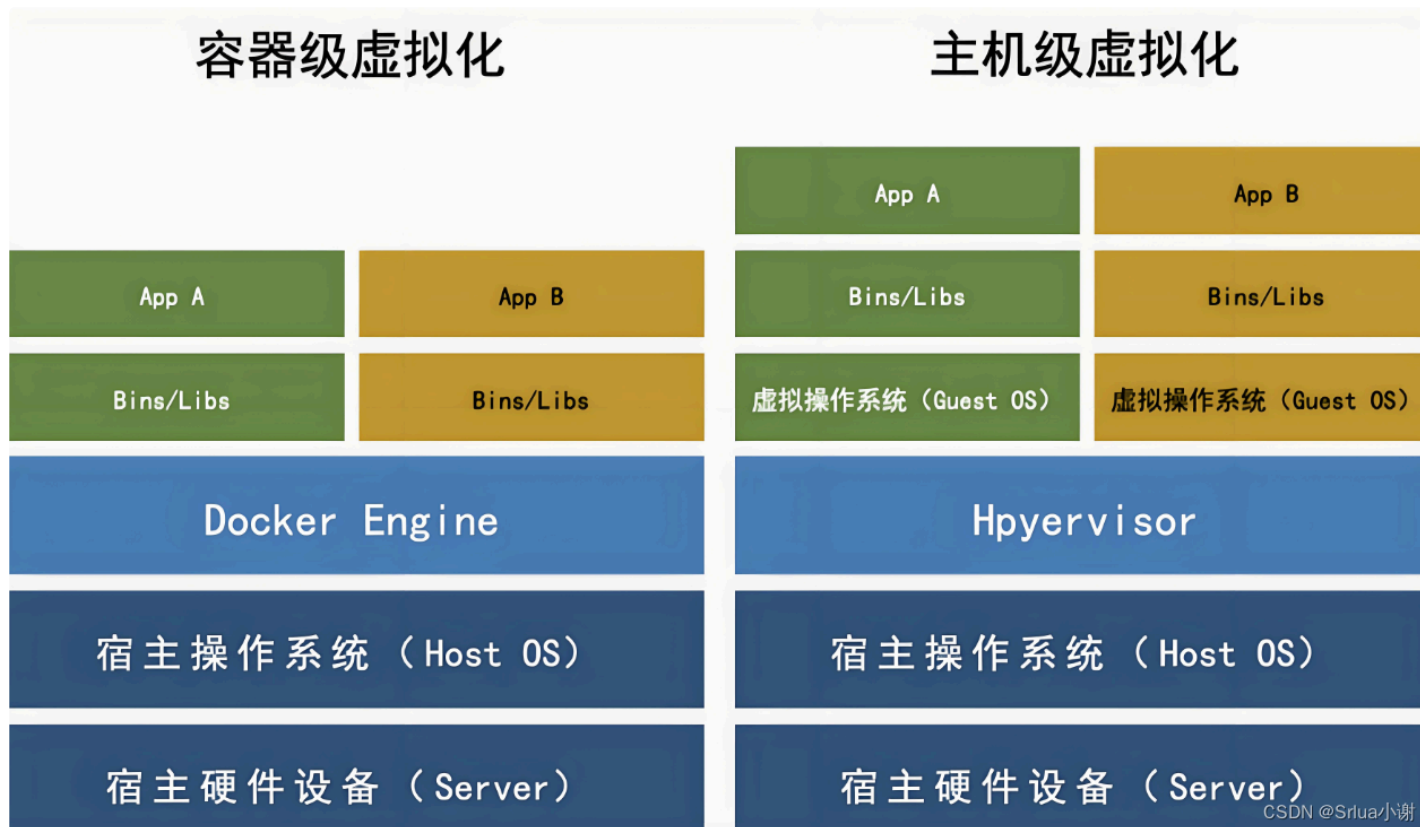
一、容器的基本概念

1.1 什么是容器？

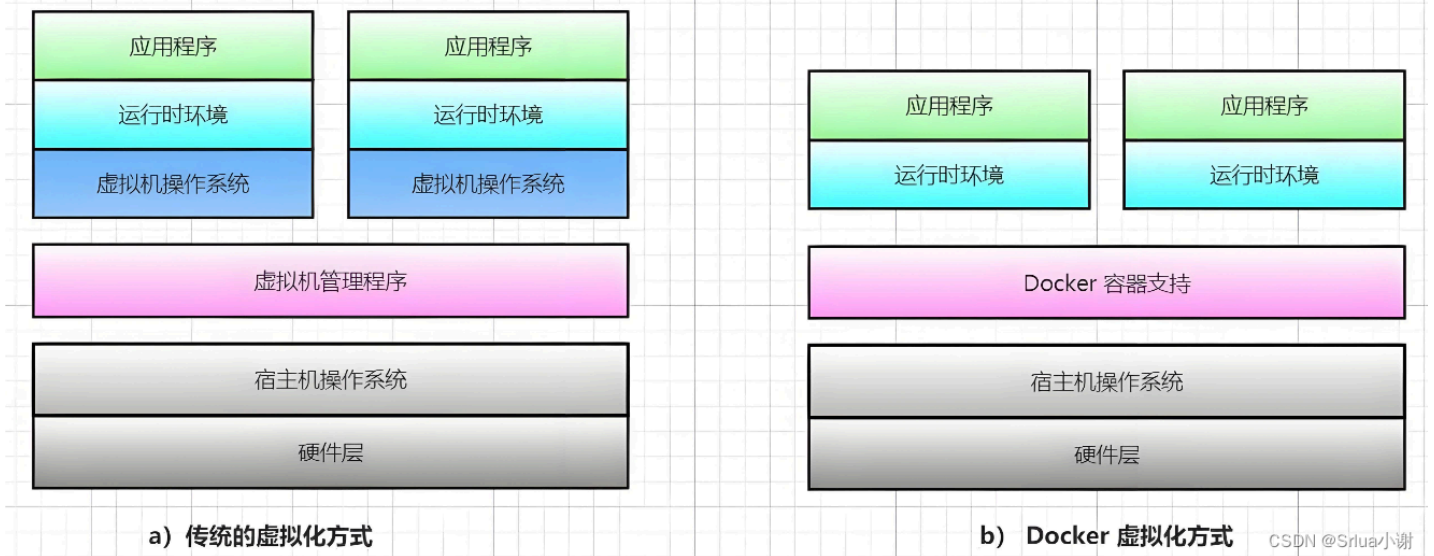


容器是一种轻量且可移植的 **虚拟化** 技术，它将应用程序及其相关的依赖项打包在一个独立的、可携带的单元中。容器内包含能顺利执行应用程序的必要元素，如代码、环境变量、进程和执行时环境以及软件依赖性等。容器类似于运输业使用的货柜，可以隔离不同的货物以便运输，容器则隔离应用程序及其依赖，以便在不同环境中运行。

1.2 容器与虚拟机的区别



Docker和传统的虚拟机方式的不同之处



虽然容器和 **虚拟机** 都用于隔离环境和提高资源利用率，但它们之间有几个关键区别：

架构差异：虚拟机包含完整的操作系统实例，运行在虚拟硬件上。容器则共享主机操作系统内核，只包含应用程序和其运行所需的依赖。

资源消耗：由于虚拟机需要运行完整的操作系统，通常比容器消耗更多的系统资源。容器更轻量级，启动速度更快。

启动时间：虚拟机启动需要数分钟，而容器通常可以在几秒钟内启动。

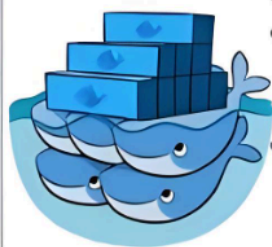
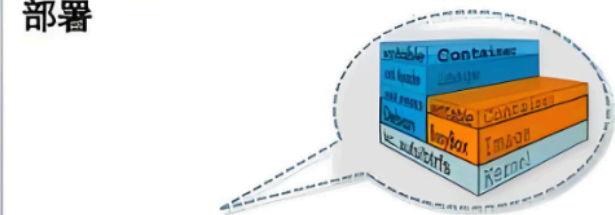
二、容器的优点

2.1 便携性

容器打包了应用程序及其所有依赖，使其可以在任何支持容器的平台上运行，无需担心环境差异。

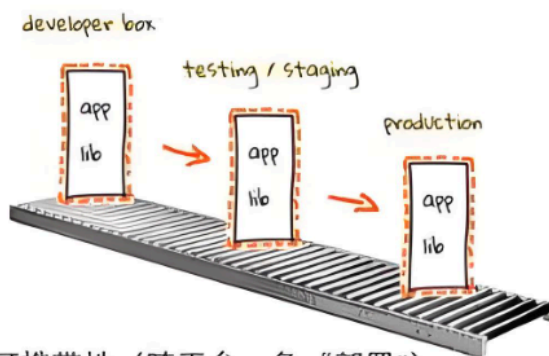
Docker加速容器技术的发展

Docker提供软件应用的集装箱，创建即部署



- 集装箱式应用管理环境。
- image分层技术：多副本&重用，节省空间、分发迅速。
- 模块化打包、快速部署、标准化管理，独立&隔离运行。

Docker彻底改变程序的交付方式



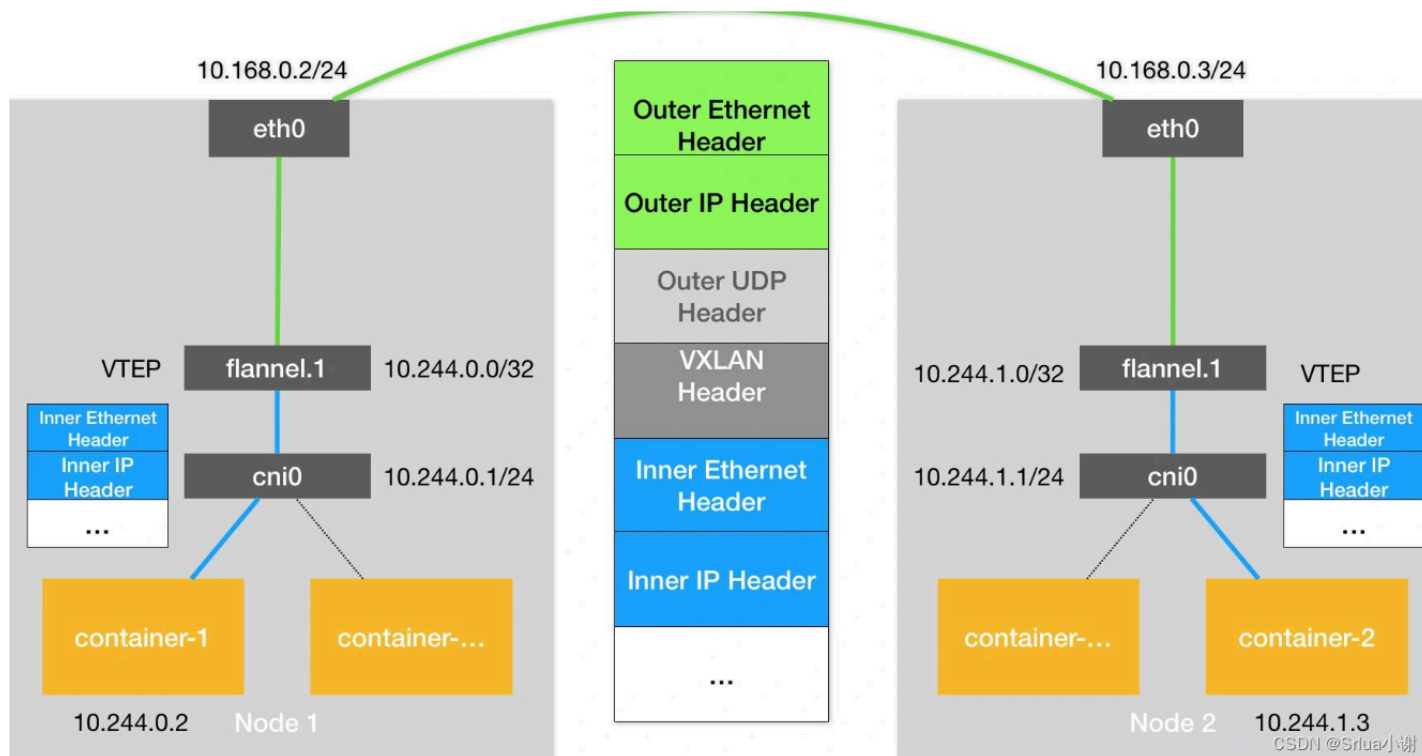
- 可携带性（跨平台、免“部署”）
- 一致性（开发&测试&生产）
- 快速分发、复制
- 轻量、隔离、无环境依赖

Docker的核心价值：构建标准化(dockerfile)、交付形态标准化（容器 & Image）、运行环境标准化（Engine）。

CSDN @Srlua小谢

2.2 隔离性

每个容器运行在独立的环境中，不会相互干扰。这种隔离性保证了安全性和稳定性。

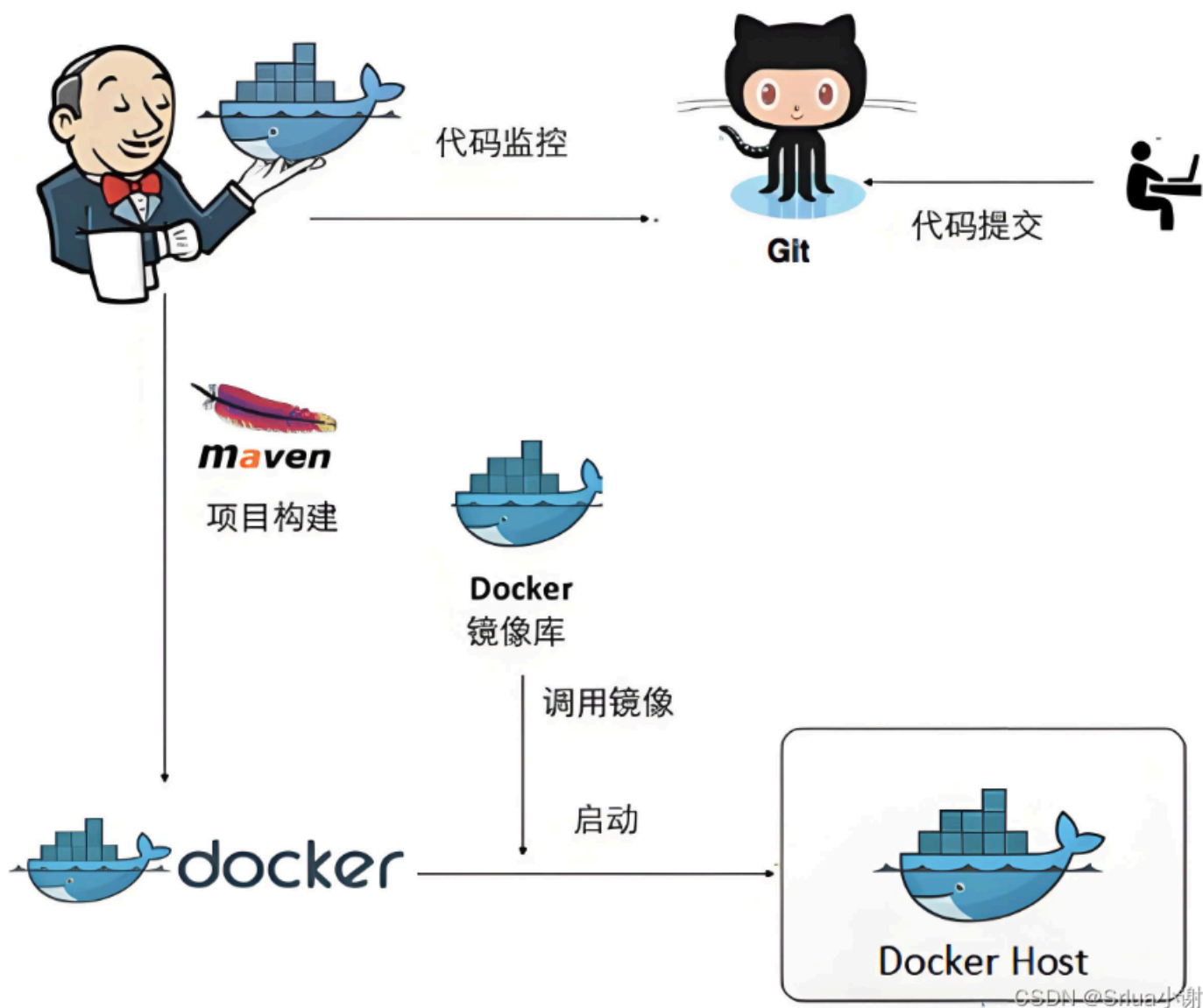


2.3 可伸缩性

容器可以快速创建和销毁，使得扩展和缩减应用变得简单而高效。

2.4 一致性

开发、测试和生产环境中的一致性是容器的一个重要优点，保证了代码在开发和生产环境中表现一致。

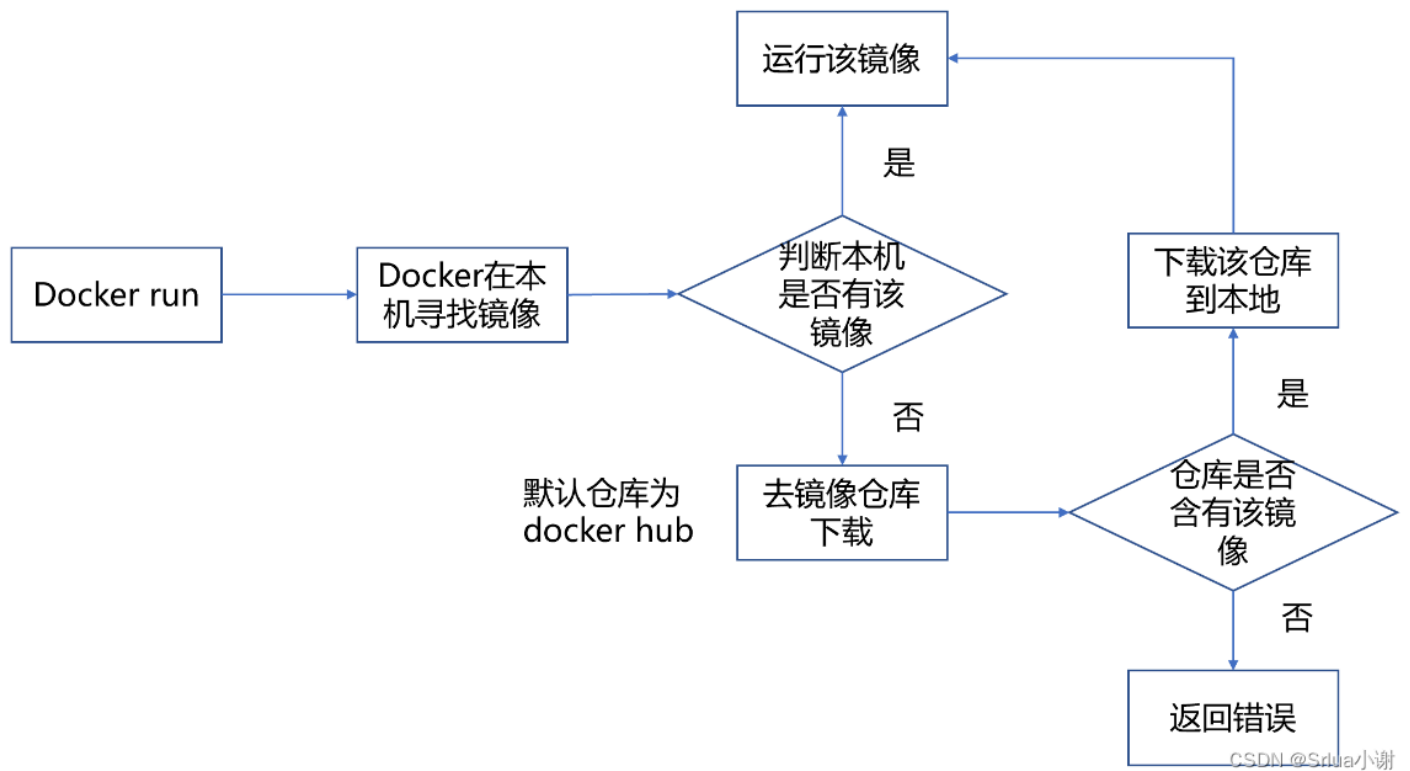


三、容器的使用

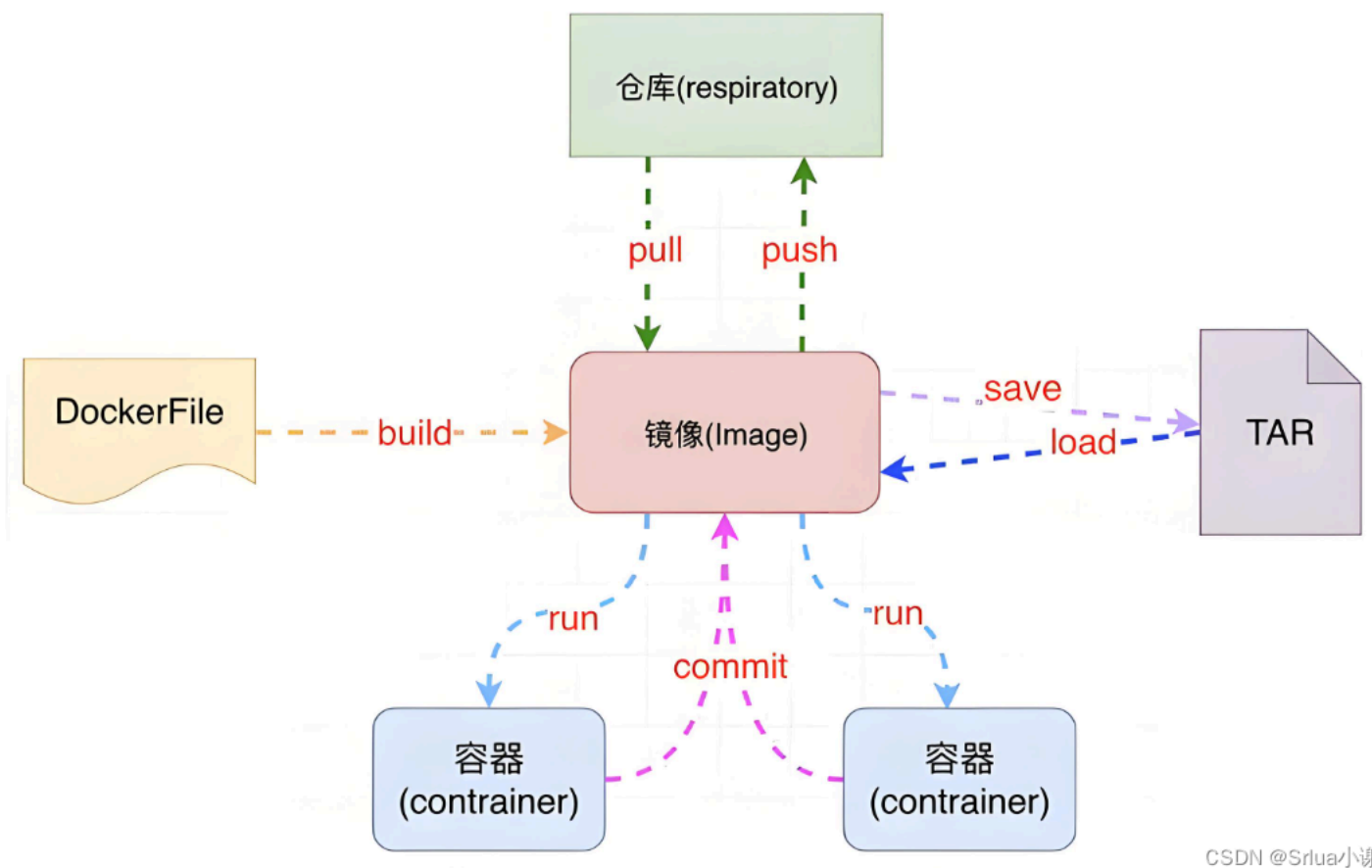
3.1 容器化应用

将应用程序容器化涉及以下几个步骤：

1. **编写Dockerfile**：定义如何构建容器镜像，包括基础镜像、安装依赖、复制代码等。



1. **构建镜像**: 使用 `docker build` 命令构建容器镜像。



1. **运行容器**: 使用 `docker run` 命令启动容器。

Dockerfile 指令	指令简介
FROM	Dockerfile 除了注释第一行必须是 FROM ，FROM 后面跟镜像名称，代表我们要基于哪个基础镜像构建我们的容器
RUN	RUN 后面跟一个具体的命令，类似于 Linux 命令行执行命令
ADD	拷贝本机文件或者远程文件到镜像内
COPY	拷贝本机文件到镜像内
USER	指定容器启动的用户
ENTRYPOINT	容器的启动命令
CMD	CMD 为 ENTRYPOINT 指令提供默认参数，也可以单独使用 CMD 指定容器启动参数
ENV	指定容器运行时的环境变量，格式为 key=value
ARG	定义外部变量，构建镜像时可以使用 build-arg <varname>=<value> 的格式传递参数用于构建
EXPOSE	指定容器监听的端口，格式为 [port]/tcp 或者 [port]/udp
WORKDIR	为 Dockerfile 中跟在其后的所有 RUN、CMD、ENTRYPOINT、COPY 和 ADD 命令设置工作目录

3.2 Dockerfile 示例

代码语言： javascript 代码运行次数： 0  运行 AI代码解释

```
# 基础镜像
FROM python:3.8-slim

# 设置工作目录
WORKDIR /app

# 复制代码到容器中
COPY . .

# 安装依赖
RUN pip install -r requirements.txt

# 启动应用
CMD ["python", "app.py"]
```

3.3 构建和运行容器

代码语言： javascript 代码运行次数： 0  运行 AI代码解释

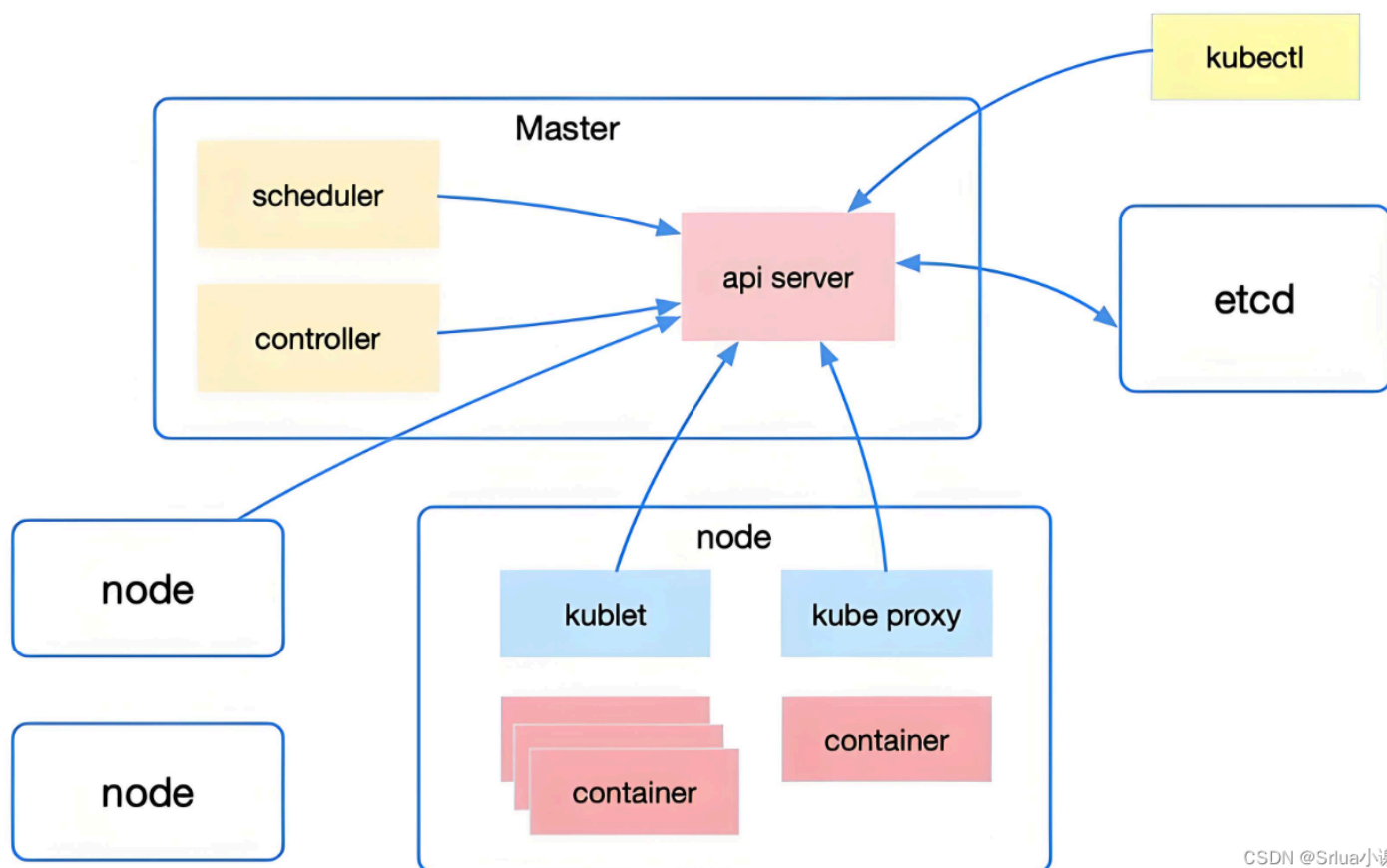
```
# 构建镜像
docker build -t my-python-app .

# 运行容器
docker run -d -p 5000:5000 my-python-app
```

四、容器编排

随着应用程序的复杂度增加，管理多个容器变得复杂。容器编排 工具，如 Kubernetes ，可以帮助管理容器的部署、扩展和网络连接。

4.1 Kubernetes 基本概念



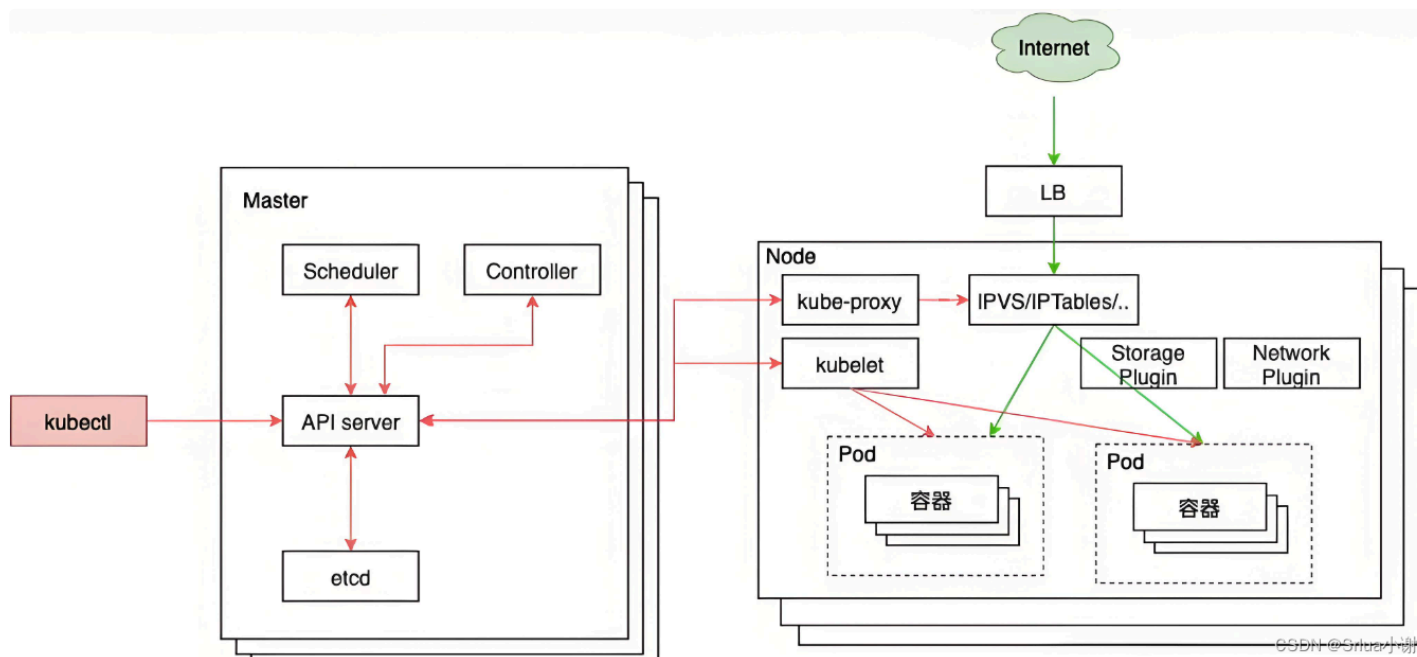
CSDN @Srlua小谢

Pod: Kubernetes中最小的部署单位，包含一个或多个容器。

Service: 定义了一组Pod的访问策略和 [负载均衡](#) 。

Deployment: 管理Pod的副本集，提供滚动更新和回滚功能。

4.2 使用Kubernetes部署应用



CSDN @Srlua小谢

1.编写Deployment配置文件

代码语言: javascript

代码运行次数: 0

运行

AI代码解释

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-python-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: my-python-app
  template:
    metadata:
      labels:
        app: my-python-app
    spec:
      containers:
        - name: my-python-app
          image: my-python-app:latest
          ports:
            - containerPort: 5000
```

2.部署应用

代码语言: javascript

代码运行次数: 0

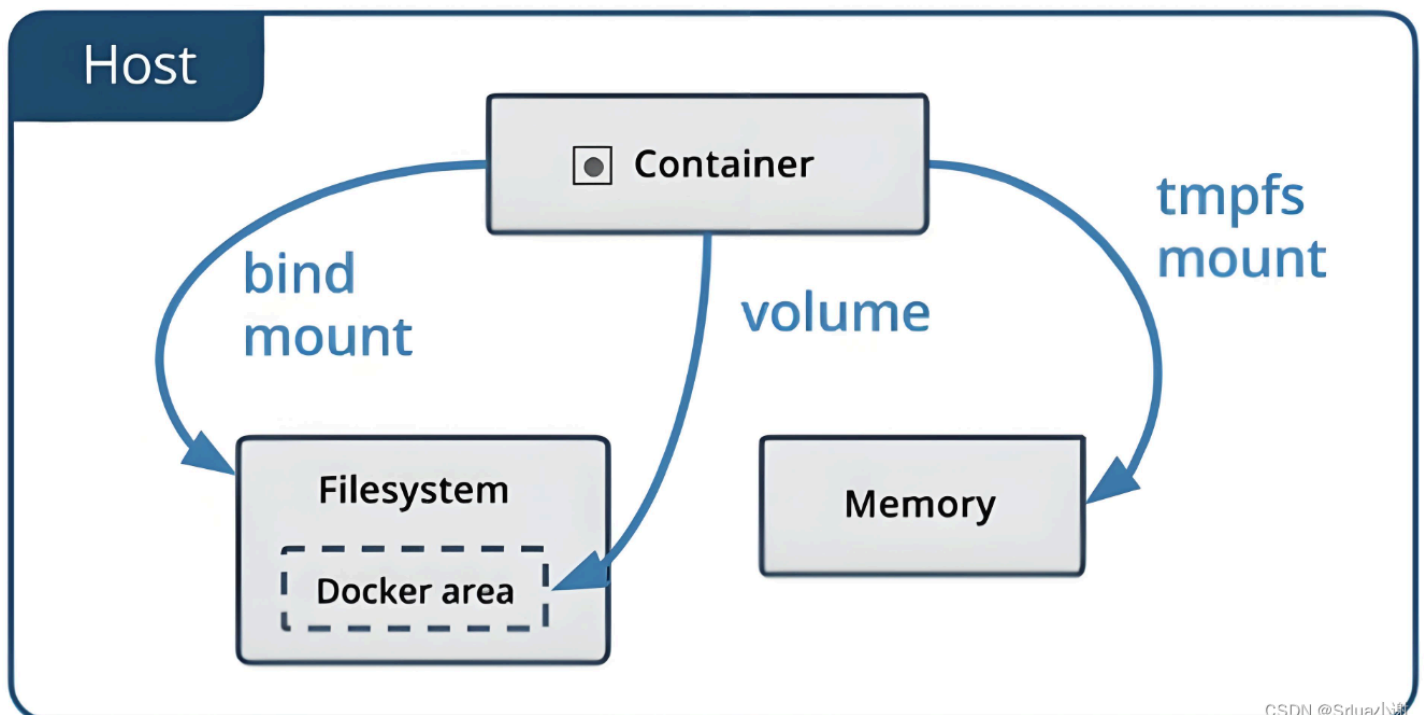
运行

AI代码解释

```
kubectl apply -f deployment.yaml
```

五、高级应用

5.1 持久化存储



容器的短暂性意味着其数据在容器删除后会丢失。通过使用持久化存储，可以保留数据。

代码语言: javascript

代码运行次数: 0

运行

AI代码解释

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

5.2 安全性

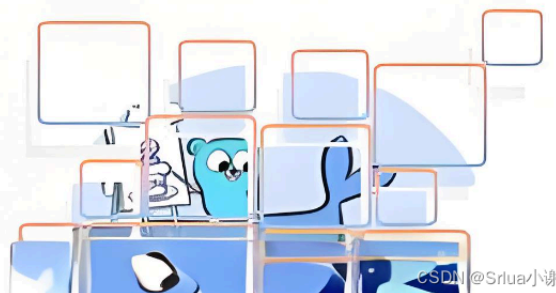
Docker存在的安全问题

■ Docker自身漏洞

- CVE官方记录Docker历史版本共有超过20项漏洞

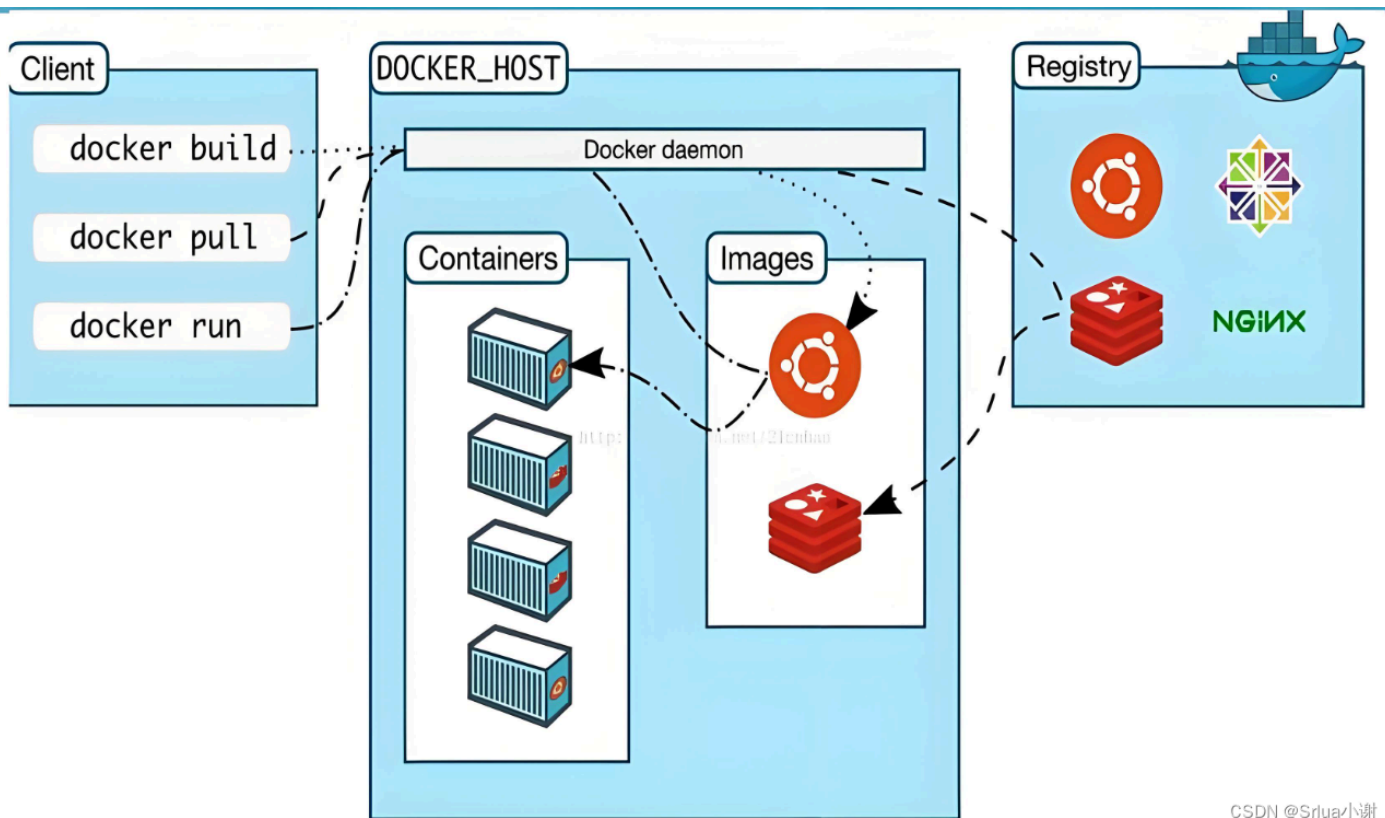
■ Docker源码问题

- 黑客上传恶意镜像
- 镜像使用有漏洞的软件
- 中间人攻击篡改镜像



容器提供了一些安全机制，如名字空间和控制组，但额外的安全措施也很重要，例如使用只读 [文件系统](#) 、限制容器权限和使用安全镜像。

5.3 服务网格

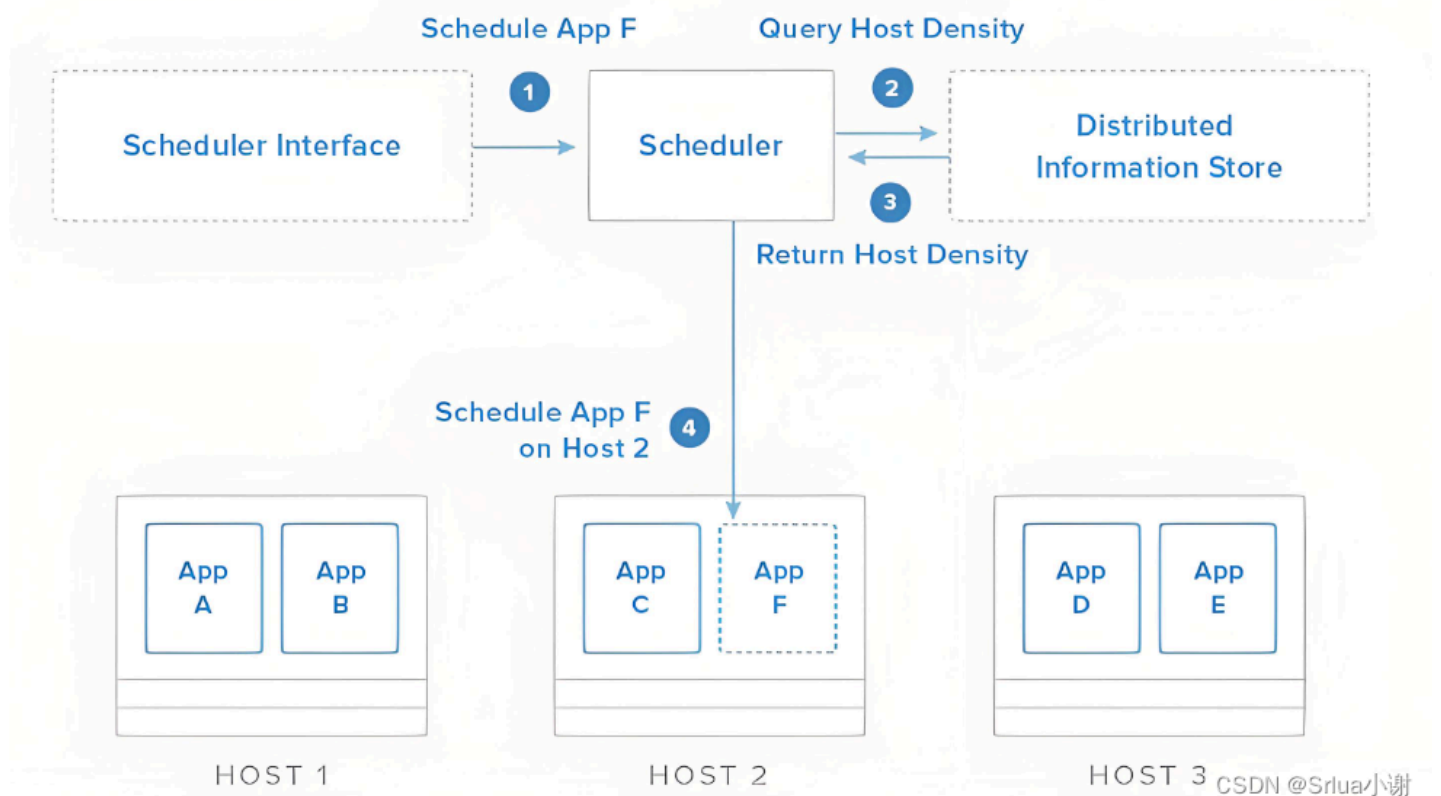


服务网格（如 Istio）为容器化应用提供了高级的网络管理功能，如负载均衡、服务发现、故障恢复和安全策略。

六、容器生态系统

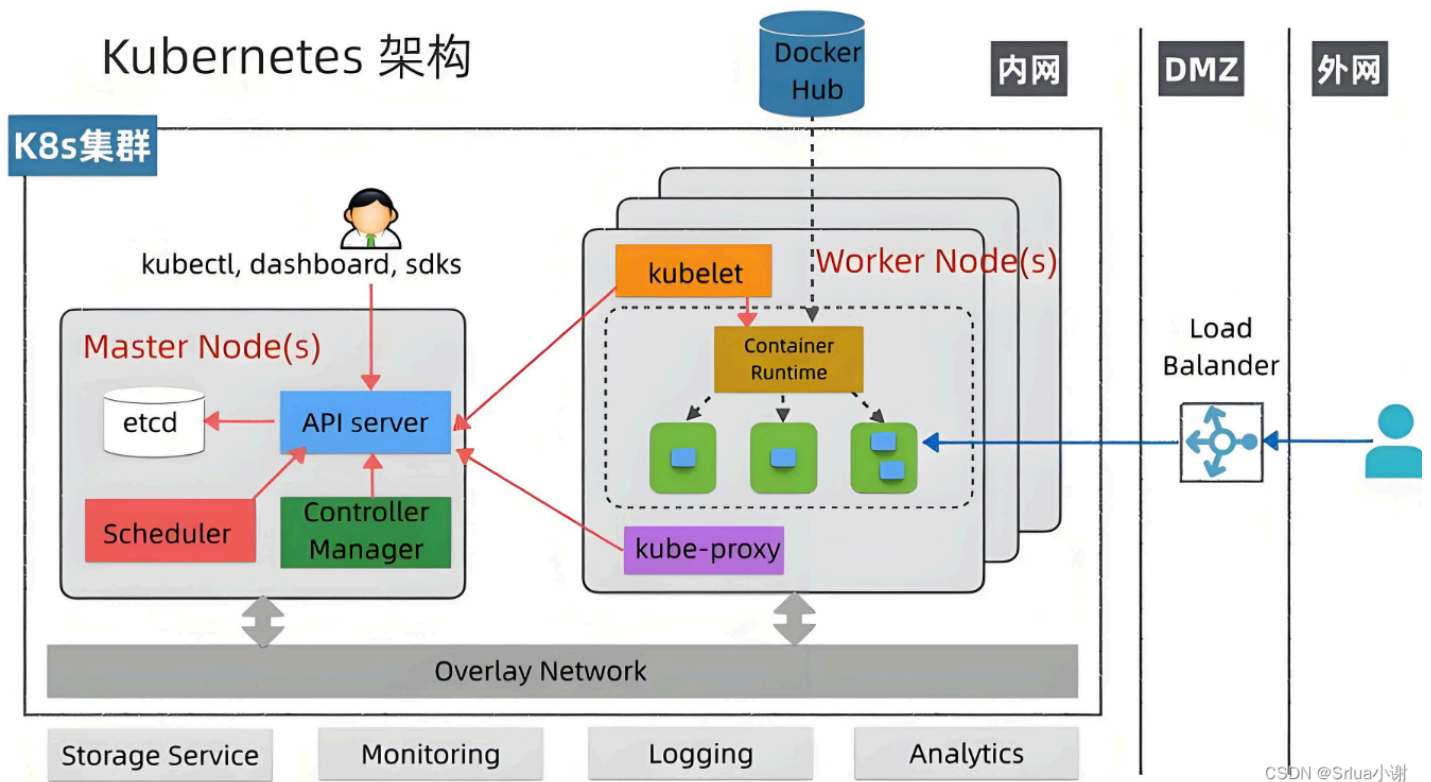
6.1 Docker

EXAMPLE: SCHEDULE APP F



Docker 是最广泛使用的容器平台，提供了容器化应用的构建、运行和分发工具。

6.2 Kubernetes



Kubernetes是一个强大的容器编排工具，帮助管理容器化应用的部署、扩展和维护。

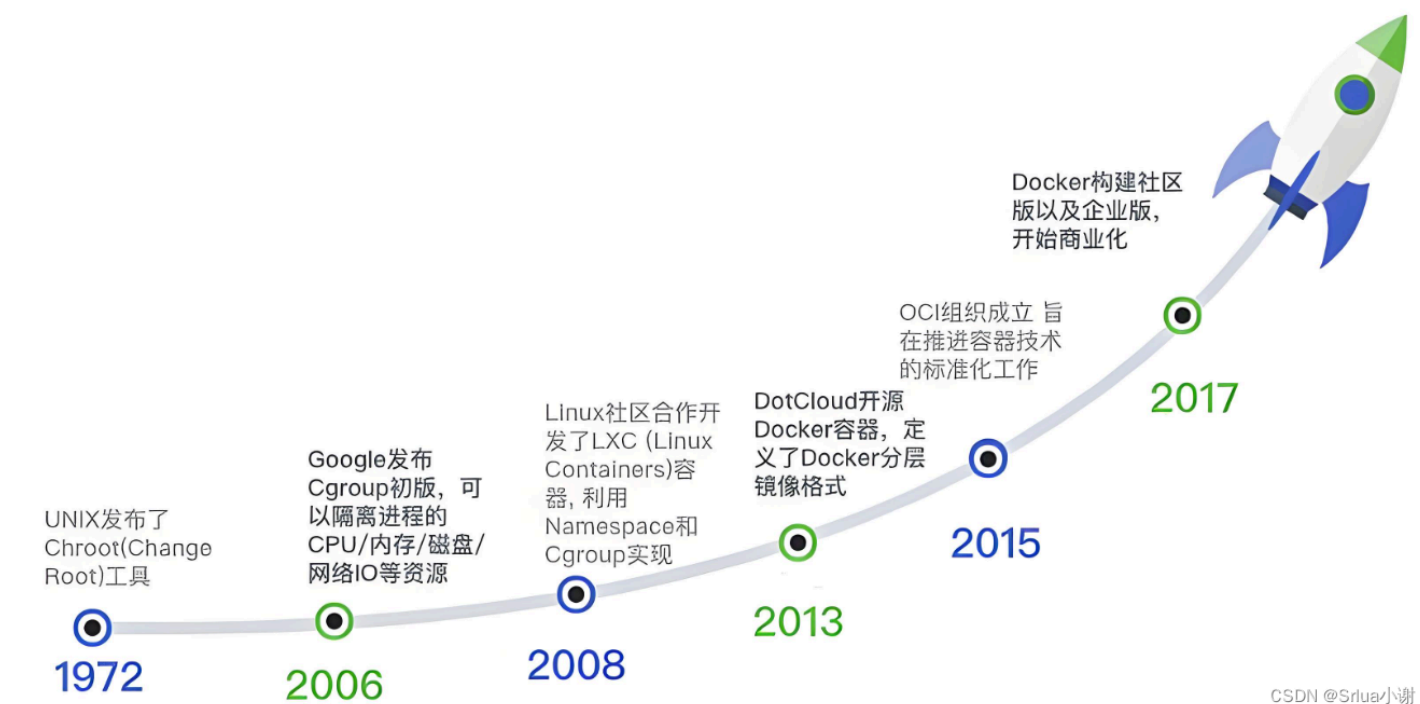
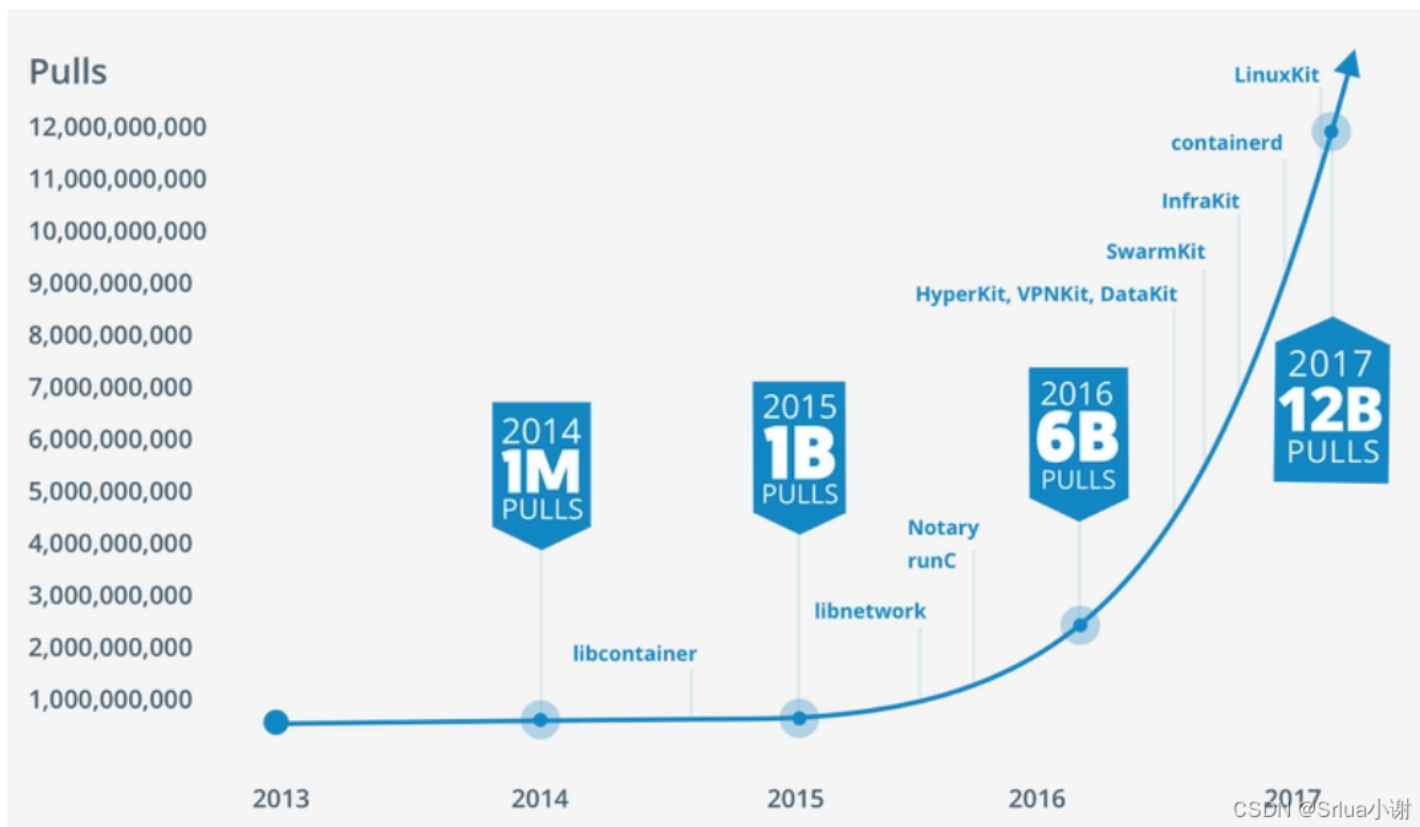
6.3 其他工具

Helm：Kubernetes的包管理工具，用于定义、安装和升级复杂的Kubernetes应用。

Prometheus：监控和告警工具，常与Kubernetes集成使用。

Grafana：[数据可视化](#) 工具，与Prometheus配合使用，提供监控数据的图形展示。

七、容器的未来发展



容器技术仍在快速发展, 未来可能会有更多的创新和改进。随着 [云原生应用](#) 的普及, 容器将成为主流的应用部署和管理方式。

总结

容器是一种强大的技术, 提供了便携性、隔离性和可伸缩性等诸多优点。通过使用Docker和Kubernetes等工具, 开发者可以简化应用程序的开发、测试和部署过程。虽然容器技术涉及的内容广泛且复杂, 但掌握其基础概念和基本使用方法将大大提升开发和运维的效率。

希望本文为您提供了一个全面的容器技术入门指南, 助您在实际项目中成功应用容器化方法。



希望对你有帮助！加油！

若您认为本文内容有益，请不吝赐予赞同并订阅，以便持续接收有价值的信息。衷心感谢您的关注和支持！

本文参与 [腾讯云自媒体同步曝光计划](#)，分享自作者个人站点/博客。

原始发表：2024-05-27，如有侵权请联系 cloudcommunity@tencent.com 删除